

ForensiX AI

An AI-Powered Digital Memory Search Engine and Cybersecurity Investigation Platform

Consolidated Project Reference Document

Summary · Objectives · Scope · Methodology · Pros & Cons · References

1. Project Summary

ForensiX AI is a desktop application designed to continuously monitor a single Windows endpoint and collect security-relevant events — Windows Event Logs, browser history, file activity, USB device events, and Windows Defender alerts — into one centralized, chronologically ordered database. An AI-based correlation engine links related events across these sources into a coherent incident timeline, while a natural-language query interface allows users without cybersecurity expertise to investigate their system using plain-English questions. To ensure the integrity of collected evidence, the platform implements hash-chained, tamper-evident logging, where each log entry's hash incorporates the previous entry's hash — making any post-collection alteration or deletion detectable. A personal behavioral baseline module learns each user's normal activity patterns and flags meaningful deviations, and an automated report generator produces structured, hash-verified incident summaries. All of this is presented through a single investigator dashboard.

The project's core motivation is that individuals and small organizations currently have no affordable, expert-free way to investigate security incidents on their own machines — existing SIEM and forensic tools are built for enterprise environments with dedicated security teams. ForensiX AI is scoped specifically to close that gap for a single endpoint, within an 8-month academic timeline.

2. Problem Statement

Modern computer systems continuously generate large volumes of security-relevant data — Windows Event Logs, browser history, file activity, USB device events, and antivirus alerts — yet this information exists in fragmented silos, each requiring a separate tool and a separate investigation process to examine.

Even when this data is accessible, it is presented in a highly technical, machine-oriented format. As a result, meaningful investigation demands specialized cybersecurity expertise, placing effective self-investigation out of reach for individual users and small organizations without dedicated security teams.

Compounding both problems is a deeper issue of trust: once collected, logs carry no inherent guarantee of authenticity. Entries can be silently modified or deleted after an incident, leaving investigators with no reliable way to confirm that the evidence they rely on is complete and unaltered.

A system is therefore needed that unifies these fragmented sources into a single timeline, enables investigation through natural language rather than technical analysis, and cryptographically guarantees that stored evidence remains tamper-evident throughout the investigation process.

3. Objectives

- **Collection** — Design a unified data collection framework capturing Event Logs, browser history, file activity, USB events, and Defender alerts in real time from a single endpoint.
- **Centralization** — Develop a centralized, chronologically ordered database that normalizes heterogeneous log formats into one schema.
- **Correlation** — Implement an AI-based correlation engine that links related events across sources into a coherent incident timeline.
- **Querying** — Build a natural-language query interface so non-experts can investigate the system using plain-English questions.

- **Reporting** — Develop an automated, AI-generated incident report module that summarizes findings in clear, human-readable language.
- **Integrity** — Implement hash-chained, tamper-evident logging so any post-collection alteration or deletion is detectable.
- **Verification** — Design a routine that checks the hash chain on demand and flags the exact point of tampering, if present.
- **Baselining** — Develop a personal behavioral baseline model that learns normal user activity and flags meaningful deviations, adapting over time.
- **Visualization** — Build a consolidated investigator dashboard presenting the timeline, alerts, and reports in one interface.

4. Scope

In Scope

- Real-time monitoring and collection of Windows Event Logs, browser history, file activity, USB device events, and Windows Defender alerts from a single Windows endpoint
- Centralized storage of all collected events in a unified, chronological database
- AI-based correlation engine to link related events into a coherent incident timeline
- Natural-language query interface for investigating the system using plain English questions
- Hash-chained, tamper-evident logging to verify log integrity
- Personal behavioral baseline module to detect deviations from a user's normal activity patterns
- Automated, structured incident report generation referencing hash-verified log integrity
- Admin/investigator dashboard for viewing timelines, alerts, and reports

Out of Scope — and Why

- **Disk imaging / bit-level acquisition** — requires write-blockers and raw sector-level file system handling; already well solved by tools like Autopsy/FTK.
- **Deleted file recovery / data carving** — requires deep NTFS/MFT internals and signature-based carving; a specialized sub-field on its own, high risk of low accuracy within the timeline.
- **Memory (RAM) forensics** — requires parsing undocumented in-memory structures (frameworks like Volatility took years to mature); a different skill set from the project's log/NLP focus.
- **Network-wide / multi-device monitoring** — would require a client-agent-server architecture and distributed systems complexity; this is what enterprise SIEM tools already do at scale.
- **Legal admissibility guarantees** — admissibility is a jurisdiction-specific legal determination, not a software claim; no tool can self-certify this.

5. Methodologies Discussed — Pros and Cons

5.1 Centralized Multi-Source Log Collection

Approach: Dedicated collectors per source (win32evtlog for Event Logs/Defender, Watchdog for file activity, WMI for USB events, direct SQLite read for browser history), normalized into one common schema and written to a central database.

Pros:

- Uses mature, well-documented Windows APIs — low technical risk
- Single query surface for all five sources instead of five separate tools
- Enables cross-source correlation, which is impossible with siloed logs

Cons:

- Browser history file is locked while the browser runs — requires careful read handling
- Five concurrent collectors writing to one database risks write-lock contention (needs a write-queue design)
- Continuous background monitoring adds CPU/disk overhead

5.2 Natural-Language Query via Local LLM

Approach: A locally run LLM (via Ollama) interprets plain-English questions and retrieves/summarizes relevant events from the centralized database, rather than the user writing structured queries.

Pros:

- Removes the need for forensic/technical expertise to investigate the system
- Local inference keeps all data on-device — no cloud exposure of sensitive logs
- Reuses an existing pre-trained model — no need to train an LLM from scratch, keeping this feasible in-scope

Cons:

- Local inference is resource-heavy — needs adequate RAM (small quantized models recommended on 8–16GB machines)
- Smaller local models may be less accurate than large cloud-hosted models
- Response latency can be a demo risk if hardware is under-provisioned

5.3 Hash-Chain Tamper-Evident Logging (not full blockchain)

Approach: Each new log entry's SHA-256 hash is computed from its own content plus the previous entry's hash, forming a verifiable chain — the core integrity principle of blockchain, without distributed consensus or multi-node infrastructure.

Pros:

- Very lightweight to implement — Python's hashlib, roughly 1–2 weeks of work
- Any alteration or deletion breaks the chain and is automatically detectable
- Avoids the infrastructure overhead of full blockchain (no nodes, no consensus mechanism needed)

Cons:

- Provides tamper-evidence, not tamper-proof distributed custody like true blockchain systems

- Must be clearly labeled as hash-chaining (not "blockchain") to withstand technical scrutiny in a viva
- If the entire local database is compromised/deleted, the chain itself offers no recovery — only detection

5.4 Personal Behavioral Baseline Modeling

Approach: The system observes a user's activity for an initial period (cold start), builds a rolling baseline of normal patterns (active hours, common apps, regular domains), and flags deviations using a confidence/severity score rather than a binary alert.

Pros:

- Personalized detection catches anomalies generic signature-based tools would miss
- Rolling baseline adapts to legitimate behavior changes, reducing alert fatigue
- Confidence scoring avoids over-alerting on minor, harmless deviations

Cons:

- Requires a cold-start period before it becomes reliable (rule-based detection needed in the interim)
- Risk of false positives/negatives if the training window is too short or unrepresentative
- Adds model-maintenance complexity compared to static signature matching

6. Research Papers Referenced

Organized by which module of ForensiX AI each paper supports. All links below are open access (arXiv, MDPI, or open journals) unless noted.

6.1 Centralized Log Collection / SIEM Architecture

Reference	Focus / Relevance	Link
GraphWeaver: Billion-Scale Cybersecurity Incident Correlation	SIEM as a centralized platform correlating security telemetry from multiple sources; supports the correlation engine design.	https://arxiv.org/pdf/2406.01842
Peacock: UEFI Firmware Runtime Observability Layer	Describes SIEM-style centralized collection and cross-source correlation for detection and response.	https://arxiv.org/pdf/2601.07402
SecureLog: Open-Source SIEM for Threat Detection & Incident Response	Open-source SIEM architecture for data collection and real-time	https://mgesjournals.com/ijstrtm/article/view/5990

Reference	Focus / Relevance	Link
	correlation, at a scale closer to this project.	

6.2 Endpoint / Desktop Cybersecurity & Forensics

Reference	Focus / Relevance	Link
Threat identification-based collection of forensic data from endpoint devices	Describes mechanisms for collecting and correlating forensic data from endpoint (desktop) devices with dashboards — close structural match to ForensiX AI's design.	https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/11750663
Digital Forensic Investigation of the ChatGPT Windows Application	Desktop application forensic analysis methodology on Windows — relevant technique reference for endpoint artifact analysis.	https://arxiv.org/pdf/2505.23938
CloudMe Forensics: A Case of Big-Data Investigation	Endpoint/desktop client forensic artifact analysis (install, login, sync activity) — relevant to file/activity log interpretation.	https://arxiv.org/pdf/1807.10218

6.3 USB & File Activity Monitoring

Reference	Focus / Relevance	Link
Forensic analysis of bad USB attacks	Methodology for detecting and mitigating malicious USB device activity — supports the USB event collector module.	https://www.edelweisspublications.com/
Forensic log-based detection for keystroke injection (BadUSB) attacks	Log-based detection approach for USB-based attacks, relevant to USB event logging design.	https://www.piraeus.gr/

6.4 Windows Event Logs & Behavioral Anomaly Detection

Reference	Focus / Relevance	Link
Anomaly detection in Windows event logs for ransomware identification	LSTM-based (DeepLog) model flags ransomware-like deviations in Windows Event Logs — direct template for the	https://www.researchgate.net/publication/396990748

Reference	Focus / Relevance	Link
	behavioral baseline module.	
Methods for host-based anomaly detection with deep learning	Establishes anomaly-based vs. signature-based detection theory, justifying the personalized baseline design choice.	https://dl.acm.org/doi/10.1145/3461462

6.5 LLM / NLP for Forensics and Security

Reference	Focus / Relevance	Link
Towards a standardized methodology for evaluating LLM-based digital forensic timeline analysis	Evaluates LLM Q&A performance on forensic timelines — the closest academic match to the NLP query module.	https://www.sciencedirect.com/science/article/pii/S2666281725000038
ForensicLLM: A local large language model for digital forensics	Fine-tuned local LLM for offline, privacy-preserving forensic analysis — supports the local-inference design decision.	https://www.sciencedirect.com/science/article/pii/S2666281725000113
AI-Augmented SOC: A Survey of LLMs and Agents for Security Automation	Surveys LLM-driven log summarization, triage, and report generation in security operations.	https://www.mdpi.com/2624-800X/5/4/95

Reference	Focus / Relevance	Link
Toward Autonomous SOC Operations: End-to-End LLM Framework	LLM-driven query generation and threat resolution — close match to the NLP query interface concept.	https://arxiv.org/pdf/2604.27321
Advancing Web Browser Forensics: Critical Evaluation of Tools	Evaluates browser forensic tools and techniques — supports the browser history collector module.	https://arxiv.org/abs/2410.12605

6.6 Blockchain / Tamper-Evident Logging

Reference	Focus / Relevance	Link
B-CoC: A Blockchain-based Chain of Custody for Evidence Management	Ethereum-based chain of custody ensuring auditable evidence integrity — the conceptual basis for the hash-chaining module, at lighter weight.	https://arxiv.org/pdf/1807.10359
SoK: Blockchain Solutions for Forensics	Systematization of multiple blockchain-based forensic evidence approaches — supports justifying hash-chaining as a lightweight alternative.	https://arxiv.org/pdf/2005.12640
Blockchain-based chain of custody: Towards real-time tamper-proof evidence management	Original blockchain chain-of-custody framework (ARES 2020) — primary reference for tamper-evidence design.	https://dl.acm.org/doi/10.1145/3407023.3407064

6.7 Automated Reporting / Documentation of Activity

Reference	Focus / Relevance	Link
Toward robust security orchestration and automated response with agentic AI	Hyper-automation approach in SOC report/response generation — supports the automated incident report module.	https://www.mdpi.com/2078-2489/16/5/365
AI-driven threat detection and automated incident response for network security	AI-generated incident response documentation and reporting workflow, tested in a simulated network environment.	https://www.mdpi.com/2624-800X

7. Feature → Objective → Problem Traceability

Problem Gap	Objective	Feature Implemented
Fragmentation	Collection + Centralization	Five-source real-time collectors writing into one unified, chronological database
Fragmentation	Correlation	AI correlation engine linking related events across sources into one timeline
Inaccessibility	Querying	Natural-language query interface (local LLM) for plain-English investigation
Inaccessibility	Reporting	Automated, human-readable AI-generated incident reports
Trust	Integrity	Hash-chained, tamper-evident logging (SHA-256 chain)
Trust	Verification	On-demand chain verification that flags the exact point of tampering
Supporting	Baselining	Personal, adaptive behavioral baseline for anomaly detection
Supporting	Visualization	Consolidated investigator dashboard